Updating from the Repositories

How instmon checks GitHub for new versions of the suite's tools, verifies each file, stages it in the library, and installs it. It covers the two update channels, the Pi Zero 2 W build choice, the single-file update, and the Full Update that runs the whole suite in order with rollback.

STABLE TESTING FULL UPDATE ROLLBACK

SUMMARY · READ-ONLY REVIEW

The repository and the library

instmon reads the suite's files from the ASL-DVS repository on GitHub. It keeps every copy it downloads in a library folder on the node, and it installs a file from there. The node never runs a file straight from GitHub.

SETTING	VALUE BY DEFAULT	SET BY
Repository	kd8pgk/ASL-DVS	The INSTMON_GITHUB_REPO variable
Branch	main	The INSTMON_GITHUB_BRANCH variable
Channel	Stable	The Update channel control (saved in instmon's config)
Build on a Pi Zero 2 W	Fork (pi02w)	The build choice (Part 2.3)
Check time limit	240 seconds	INSTMON_GH_CHECK_LIMIT_SEC
Retries for a failed fetch	2	INSTMON_GITHUB_RETRIES
Wait for a new tool to start (Full Update)	60 seconds	INSTMON_FU_START_TIMEOUT_SEC
Stay-up watch (Full Update)	60 seconds	INSTMON_FU_STABLE_SEC
One install or uninstall run	600 seconds	INSTMON_SCRIPT_TIMEOUT_SEC

Each tool file carries a version line (for example `VERSION = "9.3.107-pi02w"`). instmon reads that line from a copy to decide whether it is newer.

Where the list of files comes from

2.1 The Stable channel (the default)

Stable reads one file from the repository, `channels/stable.json`. It lists each tool with its file name, version, git blob id, commit, the date it landed on main, the date it was promoted, and the reason for the promotion. The file is written by the repository's promote workflow, not by hand. The rules are in `channels/rules.json`: a release becomes stable once 4 newer releases of that tool are on main and it has been there 3 days, or once it has been the newest release for 14 days; cosmetic builds do not count; and a floor sets the lowest version a tool may be promoted at (instmon 2.50.0). `channels/overrides.json` can hold a version back or hotfix one to stable at once. A file that is not in the list is not offered.

If the repository has no stable list yet, the check says so, and tells the operator to switch to Testing.

2.2 The Testing channel

Testing lists the files on `main` through the GitHub contents API. It offers the latest commit, before any release has proven it. Switching to Testing needs a confirmation (without it the answer is a warning that confirmation is needed), and the message says the files come straight from main. Switching back to Stable needs none.

Changing the channel stops a check that is running, and clears the last check's results. The page must press Check GitHub again.

2.3 The Pi Zero 2 W build

On a Pi Zero 2 W, instmon offers two builds: the fork (the default, with the `-pi02w` version suffix) and the full build. On any other Pi, the choice does not apply. The choice is saved, and a change refreshes the update plan, so the next Full Update installs the chosen build of SysMon and the Dashboard.

2.4 Files are grouped by kind

Files are grouped into categories (for example the dashboard, SysMon, wifimon, 44helper, Node Talker, and the scripts). Within a category, the newest file is kept for each build variant. So a fork file and a full-build file are compared only with their own kind.

Check GitHub

3.1 What the button does

1. The button starts a check in the background. Only one check runs at a time, and a second press is refused while one is running.
2. The check reads the file list (the stable list, or the contents of main).
3. For each file, it reads the version. It downloads the file only if the library has no copy with the same blob id, and caches the version by blob id.
4. It compares each file with the library. The result is one status per file.
5. If the check runs past its time limit (240 seconds by default), the check is stopped.

A generation counter stops an old check from writing its results after the channel has changed or a new check has started.

3.2 The statuses

STATUS	MEANING
up_to_date	The library already has a copy with this exact blob id.
update	The library's newest copy of this kind is older than the file on GitHub.
missing	The library has no copy of this kind yet.
newer_local	The library's newest copy is newer than the file on GitHub.
error, or skipped	The file could not be read or was refused (for example, it has no version line, or its name has characters instmon refuses).

Only "update" and "missing" are offered for a single-file update. A file that is skipped is listed with its reason, and it is not installed.

3.3 Version comparison

Versions are compared by number, not by text, so 9.3.100 is newer than 9.3.99. A fork file and a full-build file are never compared with each other (Part 2.4).

What is downloaded, and how it is checked

- **HTTPS only.** A GitHub address that does not start with https:// is refused.
- **Size limits.** The file list has a 1 MB limit, and a file is held to the upload limit.
- **Retries.** A failed fetch is retried (two retries by default). Network errors are recognised, so a check that cannot reach GitHub is reported as a network problem, not as a bad file.
- **GitHub's limits.** A message that GitHub's hourly limit has been reached is recognised and reported. The check then stops and says to try later.
- **Download time.** A download is limited to 1,800 seconds (INSTMON_GITHUB_MAX_DOWNLOAD_SEC).
- **Blob check.** Every downloaded file is hashed in git's way (the blob id). The hash must match the id in the file list. A file that does not match is refused, and the message says the file changed during the check.
- **UTF-8 text only.** A file that is not UTF-8 text is refused when its version is read.

4.1 Staging in the library

A verified file is written to the library folder for its category, under its own name. If the library already has a different file with that name, the new file is refused and the old one is left alone. Writes use a compare-before-write, so an unchanged file is not rewritten.

The Update button on one file

A single-file update downloads the file into the library. It does not install it. The operator then presses Install on the library file, which runs the component's own `--install`.

1. The page sends the file's name with `confirm: true` to the GitHub update action. Without the confirmation, the request is refused.
2. The file must be in the last check's results. If not, the page is told to press Check GitHub first.
3. Only an "update" or "missing" file is accepted. An older file (newer_local) is accepted only on the Stable channel, and only when the operator explicitly allows older versions. This is the way to roll a tool back to an earlier stable release.
4. The file is downloaded, verified against its blob id, and written into the library (Part 4). A library file that is locked by a running install is not overwritten, and a different file of the same name is refused.
5. The message says "press Install on it to use it" (or, for a script, to run it from the library). The install itself is covered in Part 3 of this series (the install button and `--install`).

Updating the whole suite in order

6.1 The plan

The Full Update runs the check, then makes a plan for each tool in this order: 44helper, wifimon, Node Talker, Node Recorder, Pibrary, the Dashboard, SysMon, and instmon last. A tool that is not installed is left alone. A tool whose installed file has no version line is marked for a hand update. Only "update" and "switch" entries are acted on.

6.2 For each tool

1. **Save a copy** of the installed file in the library, if one is not already there.
2. **Uninstall** the old version with its `--uninstall`. A tool whose installed file has no `--uninstall` is skipped and left alone. If the uninstall exits with an error, the install goes ahead anyway, and the log says so.
3. **Install** the new file with its `--install`.
4. **Wait up to 60 seconds** for the tool to start and answer on its port.
5. **Watch it for 60 seconds** to make sure it stays up. instmon itself is not watched this way (Part 6.4).

The uninstall and install commands do not run inside instmon's own process. Each runs as a transient systemd unit (`instmon-fu-job`, started with `systemd-run`), with a run-time limit of 600 seconds, so the work survives an instmon restart and can be stopped by stopping the unit. Its output is read back into instmon's log. Without `systemd-run`, instmon runs the command directly.

6.3 Rollback

If a tool does not start and answer, or does not stay up, instmon puts back the saved copy with its `--install` and waits for it to answer. The outcome is one of three: the tool is rolled back (warning), the tool could not be rolled back either (it "needs attention", and is left not running), or the new version is kept.

6.4 instmon updates itself last

instmon is replaced last, because it is the program doing the work. Before its file is changed, a one-shot systemd timer (`instmon-update-guard`) is armed for 90 seconds. When it fires, it tries to open instmon's port. If that fails, it unlinks the new install and runs the saved old file's `--install`. A successful install stops the timer. If the timer cannot be set up, instmon is left alone and the Full Update says so. After the new instmon is installed, the run waits 45 seconds and reports that the new version takes effect when instmon restarts.

6.5 Controls

- **Stop** asks the run to stop after the current step. It needs a confirmation. A step that has begun is finished or rolled back.
- **Dismiss** clears the summary once the run is finished.
- **Resume at startup.** If instmon restarts during a Full Update, it resumes or recovers the job when it starts again.

6.6 After the run

When every tool is current, the run reports that. Otherwise it reports each tool's outcome. Two extra checks follow the run: the Wi-Fi menu script and the manuals. Their details were not traced in full for this part.

Apps and documents

- **Apps.** instmon fetches the Node Talker Mobile APK from the newest release whose tag matches `android-vX.Y.Z`. The file name must match `NodeTalker-X.Y.Z.apk` (a `-debug` build is also accepted by the pattern). The file is kept in the library's apps folder. The first fetch runs on a schedule, with waits of 20, 90, 180 and 360 seconds.
- **Documents.** The docs folder of the repository is listed through the contents API and fetched into the library.

Both use the same HTTPS, size and verification rules as the tool files.

What the update path shows

8.1 Two layers of trust

Stable offers only files that a workflow has promoted, and every download is checked against its git blob id. The book can describe the design as curated for most users, and checked for every user.

8.2 Rollback at every step

Each tool keeps a copy of its old file until its new one is healthy, and instmon is guarded by its own timer. The book should explain that a failed update is put back in most cases, and that the rare exception is flagged as "needs attention".

8.3 The check relies on GitHub

The check needs a working connection to GitHub, and the GitHub API has an hourly limit. A node with no connection gets a network message, not an update. The book should say the update path needs the internet.

8.4 Not checked

- The release workflow that writes the stable list (it is in the repository's CI, not in this file).
- The Wi-Fi menu and manuals steps after a Full Update.
- What the install of each component does beyond its own `--install`, which is covered in Part 3.